

## Deepsleep

1)

### Welche Vorteile bringt der Deep-Sleep-Modus des ESP32?

- **Energieeffizienz:** Im Deep Sleep verbraucht der ESP32 nur minimal Strom, was ideal für batteriebetriebene Geräte ist.
- **Flexibles Aufwecken:** Der ESP32 kann durch verschiedene Ereignisse wie Timer, externe Interrupts oder Sensoren aufwachen.
- **Speicherung von Zuständen:** Daten im RTC-Speicher (Real-Time Clock) bleiben während des Deep Sleep erhalten.

2)

### Untersuche den nachstehenden Code, welcher den ESP in den Deep Sleep versetzt und nach 5 Sekunden wieder aufwecken lässt.

- **Warum ist der Variable bootZaehler das Schlüsselwort RTC\_DATA\_ATTR vorgestellt?**
  - Das Schlüsselwort **RTC\_DATA\_ATTR** sorgt dafür, dass die Variable im **RTC-Speicher** abgelegt wird. Dieser Speicher bleibt auch während des **Deep Sleep** erhalten. Das bedeutet, dass der Wert von **bootZaehler** nach dem Aufwecken des ESP nicht verloren geht und weiterverwendet werden kann.
- **Woran lässt sich in dem Verhalten des Programms erkennen, dass der ESP tatsächlich in den Schlafmodus gegangen ist?**
  - **Keine Kommunikation stattfindet:** Es werden keine Daten über die serielle Schnittstelle gesendet.
  - **LEDs oder Peripherie sind deaktiviert:** Falls Peripheriegeräte oder LEDs an GPIOs hängen, sind diese ausgeschaltet.
  - **Stromverbrauch sinkt drastisch:** Mit einem Multimeter kannst du überprüfen, dass der Stromverbrauch erheblich reduziert ist.

3)

Schließe einen Taster an den ESP an. Programmiere den ESP32 so um, dass er anstelle über den Timer über das Betätigen eines Tasters des ESP aus dem Deep Sleep erwacht.

Der Befehl für den Timer Wake-up muss dabei durch den für den External Wake-up ersetzt werden, wobei GPIO\_NUM\_X für den Pin und level für den Zustand high oder low steht. Es kommen nur RTC IO Pins in Frage.

```
#include "esp_sleep.h"
#define WAKE_UP_PIN GPIO_NUM_33

void setup() {
    Serial.begin(115200);
    pinMode(WAKE_UP_PIN, INPUT_PULLUP);
    Serial.println("Going to Sleep");
    esp_sleep_enable_ext0_wakeup(WAKE_UP_PIN, 0);
    esp_deep_sleep_start();
    Serial.println("Error");
    delay(2000);
}

void loop() {
}
```

Die Ausgabe zeigt an, dass der Deep-Sleep-Modus beim Tastendruck aktiviert wird. Sobald die Taste wieder losgelassen wird, wacht der ESP32 auf und verlässt somit den Deep-Sleep-Modus.

4)

Erweitere dein Programm und deinen Aufbau aus Aufgabe 3 so, dass bei jedem Mal die gemessene Temperatur eines angeschlossenen Temperatursensors ausgegeben wird.

```
#include "esp_sleep.h"
#define WAKE_UP_PIN GPIO_NUM_33

int wert = 0;
double voltage = 0;
double tempC = 0;

void setup() {

    Serial.begin(115200);
    Serial.println("Start");
    pinMode(26, INPUT);
    pinMode(WAKE_UP_PIN, INPUT_PULLUP);

    Serial.println("Going to Sleep");

    wert = analogRead(26);
    voltage = ((float) wert / 4096.0) * 3300;
    tempC = (voltage - 500) * 0.1;

    Serial.println(tempC);

    esp_sleep_enable_ext0_wakeup(WAKE_UP_PIN, 0);
    esp_deep_sleep_start();

    Serial.println("Error");
    delay(2000);
}

void loop() {
}
```

Wir bekommen nun eine Ausgabe der Temperatur, beim Druck auf die Taste.